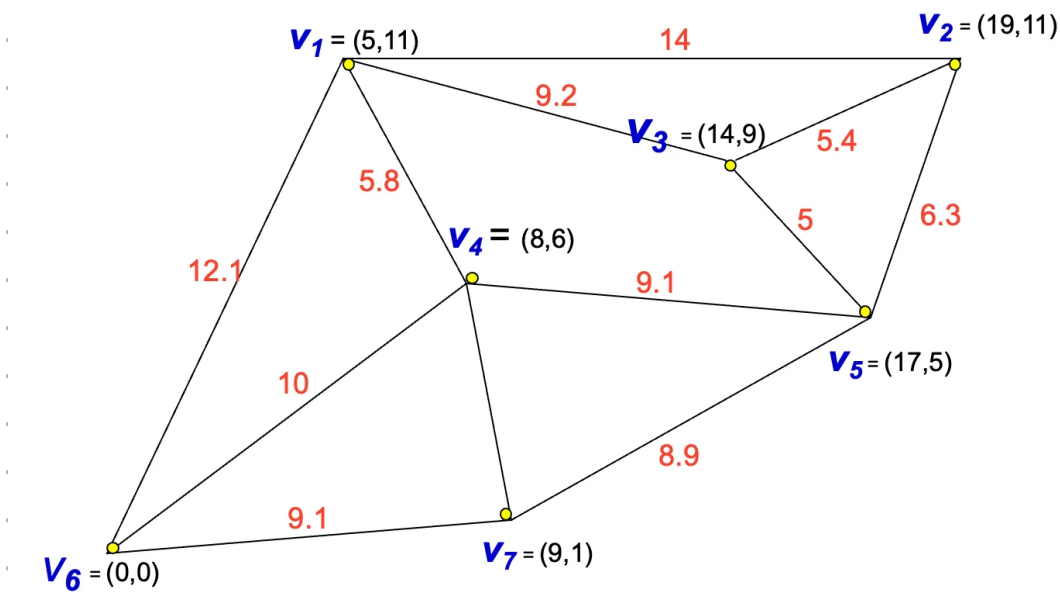


• **Euclidean Graph:** A graph where $w(e_i) = \text{Euclidean distance b/w its endpoints } v_i \text{ and } v_j$

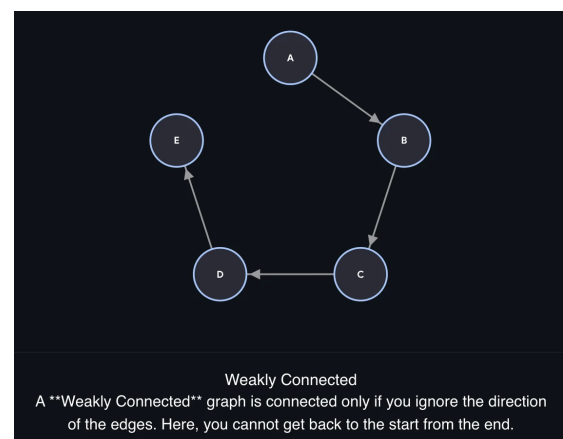
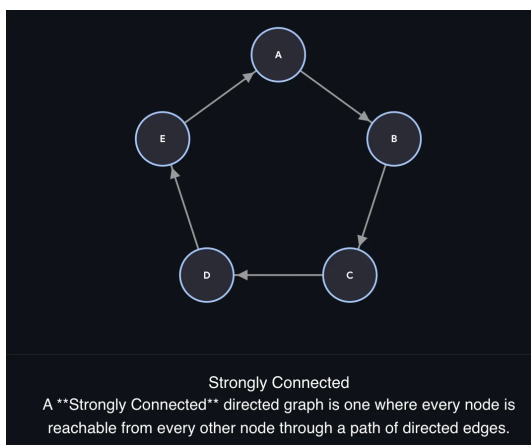


• **Simple path:** A route through a graph where you never visit the same vertex more than once.

◉ We allow a path from a vertex to itself; if this path contains no edges, then the path length is **0**.

■ If the graph contains an edge (v, v) from a vertex to itself, then the path v, v is sometimes referred to as a loop.

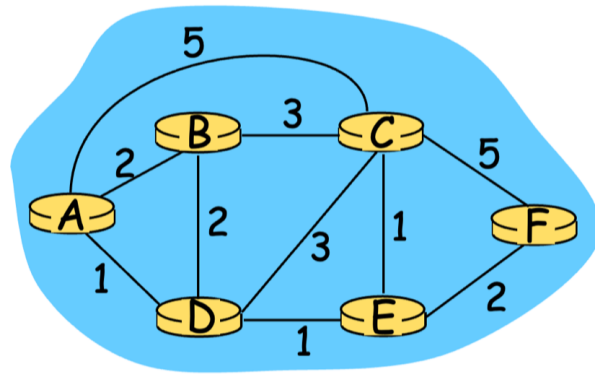
Connected graph: A path exists from every vertex to every other vertex



Complete graph: There exists an edge b/w every pair of vertices

Dijkstra's Algorithm

Step	start N	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
0	A	2,A	5,A	1,A	infinity	infinity
1	AD	2,A	4,D		2,D	infinity
2	ADE	2,A	3,E			4,E
3	ADEB		3,E			4,E
4	ADEBC					4,E
5	ADEBCF					

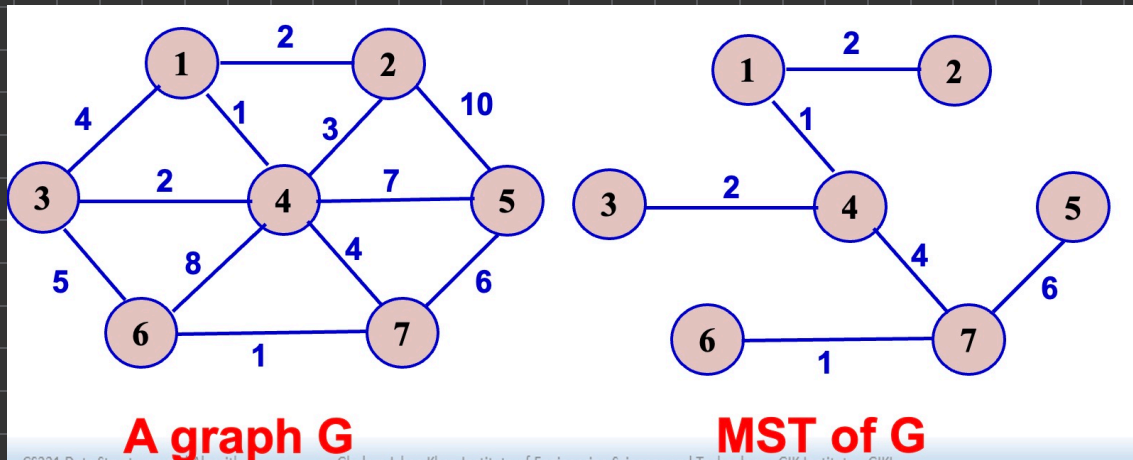


- Dijkstra's algorithm works for weighted graph to find least-cost path.
 - We can also use it on non-weighted graph by setting all weights to 1 or some values.
- This will instead give us the shortest path
- To find least-cost/shortest path from every node to all other nodes, we have to re-run Dijkstra on each one separately.

Minimum Spanning Tree

- A MST of an undirected graph G is a tree formed from graph edges that connects all vertices of G at lowest total cost.
- An MST exists if and only if G is connected & undirected.
- There can be multiple MSTs for a G if there duplicate costs.
- For unweighted graph with all weights set to 1, every spanning tree is a valid MST.

Total MSTs possible = V^{V-2}



Prim's Algorithm

```

for each  $u \in G.V$ :
     $u.key = \infty$            // Initialize all keys to infinity
     $u.\pi = NIL$              // No parent yet

 $r.key = 0$                  // Source vertex key = 0
 $Q = G.V$                   // Build min-heap on basis of keys  $O(V)$ 

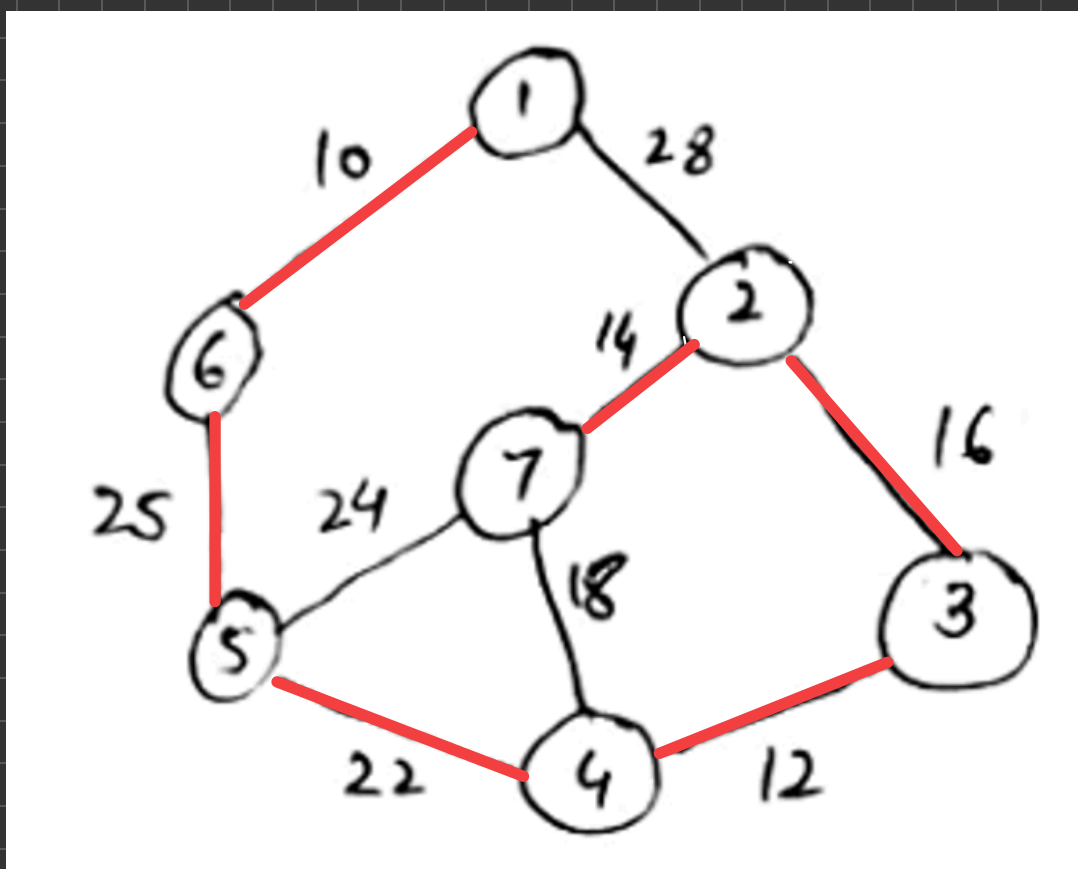
while  $Q \neq \emptyset$ :        // Repeat  $V$  times
     $u = \text{Extract-Min}(Q)$     //  $O(\log V)$ 
    for each  $v \in G.adj[u]$ :    //  $O(V-1)$  total
        if  $v \in Q$  and  $w(u,v) < v.key$ :
             $v.\pi = u$ 
             $v.key = w(u,v)$     // Decrease-Key  $O(\log V)$ 
    
```

$$D(v) = \min(D(v), w(u,v))$$

Example

~~V₁~~ ~~V₂~~ ~~V₃~~ ~~V₄~~ ~~V₅~~ ~~V₆~~ V₇

$D(V_2), p(V_2)$	$D(V_3), p(V_3)$	$D(V_4), p(V_4)$	$D(V_5), p(V_5)$	$D(V_6), p(V_6)$	$D(V_7), p(V_7)$
28, V ₁	∞	∞	∞	18, V ₁	∞
28, V ₁	∞	∞	25, V ₆	—	∞
28, V ₁	∞	22, V ₅	—	—	24, V ₅
28, V ₁	12, V ₄	—	—	—	18, V ₄
16, V ₃	—	—	—	—	18, V ₄
—	—	—	—	—	14, V ₂
—	—	—	—	—	—



Complexity Analysis:

$$V + V + V \log V + V^2 \log V$$

This is for a Dense graph. Another way to write this

is:

$$V + V + V \log V + E \log V$$

where $E = V^2$ if Dense

So,

$$O(E \log V)$$

Kruskal's Algorithm

MST-Kruskal (G,w)

F = {}

For each vertex $v \in V[G]$ — V

MakeSet(v)

sort the **edges (E)** in increasing order by weight **w** — $E \log E$

for each edge $(u,v) \in E$ — $\log E \times E$

if **FindSet(u) $\neq$ FindSet(v)**

F = F U {(u,v)}

UNION(u,v)

or make min-heap
so we get E

Extraction time

no. of edges/extractions

Example

$\{V_1\}$ $\{V_2\}$ $\{V_3\}$ $\{V_4\}$ $\{V_5\}$ $\{V_6\}$ $\{V_7\}$

$\{V_1, V_6\}$ $\{V_2\}$ $\{V_3\}$ $\{V_4\}$ $\{V_5\}$ $\{V_7\}$

$\{V_1, V_6\}$ $\{V_2\}$ $\{V_3, V_4\}$ $\{V_5\}$ $\{V_7\}$

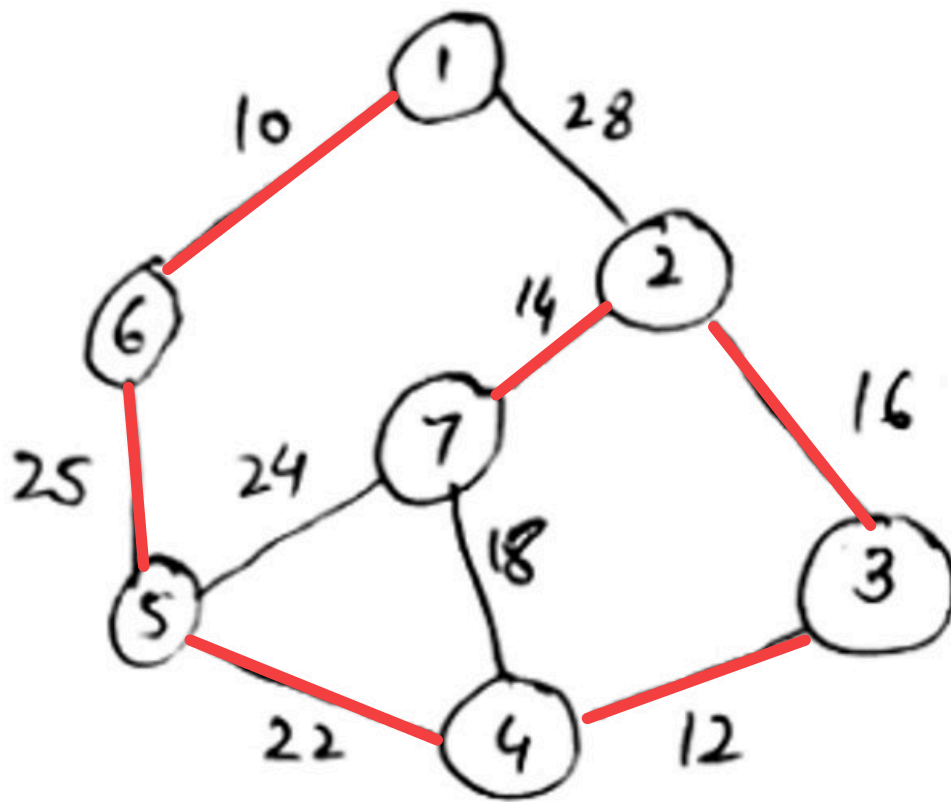
$\{V_1, V_6\}$ $\{V_2, V_7\}$ $\{V_3, V_4\}$ $\{V_5\}$

$\{V_1, V_6\}$ $\{V_2, V_3, V_4, V_7\}$ $\{V_5\}$

$\{V_1, V_6\}$ $\{V_2, V_3, V_4, V_5, V_7\}$

$\{V_1, V_2, V_3, V_4, V_5, V_6, V_7\}$

$$F = \left[(V_1, V_6), (V_3, V_4), (V_2, V_7), (V_2, V_3), (V_4, V_5), (V_5, V_6) \right]$$



Time Complexity

Build Heap: $E + (V-1) \cdot \log E$

Dense graph ($E=O(V^2)$):
 $TC = O(V^2)$

Sparse graph ($E=O(V)$):
 $TC = O(V \log V)$

$$E + E \log E$$

- Given some edges that create a cycle are discarded.

So, actual extractions become $(V-1)$ instead of E

$$E + (V-1) \log E$$

1. Prim's Algorithm: The Growing Tree (Always Connected)

Prim's algorithm starts at a single, arbitrary node and looks for the cheapest edge connected to the tree you've already built.

- Because it only ever adds edges that touch the existing tree, **the MST under construction is always one single, fully connected component** at every single step. It grows outward like a puddle spreading on the floor.

2. Kruskal's Algorithm: The Forest (Disconnected Components)

Kruskal's algorithm doesn't care about a starting node. It just sorts all the edges in the entire graph from lightest to heaviest and starts picking them blindly (as long as they don't form a cycle).

- Because it just grabs the cheapest edges from anywhere in the map, it will initially form multiple, tiny **disconnected islands** of nodes. In graph theory, a collection of disconnected trees is called a **Forest**. As the algorithm progresses, these separate islands gradually merge together until they finally form one single connected tree at the very last step.

